

CSC258 LEC04 - Numerical Representations

Unsigned Binary Integers

- Of form $x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \dots + x_12^1 + x_02^0$
- Range from 0 to $2^n - 1$
- 64-bit range is 0 to 18,446,774,073,709,551,615

Example

$$0000\ 0000 \dots 0000\ 1011_2 = 0 + \dots + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 11_{10}$$

Octal Representation

- Base 8
- Compact representation of bit strings
- 3 bits per octal digit

0	000	4	100
1	001	5	101
2	010	6	110
3	011	7	111

Example

$$7650_8 = 111\ 110\ 101\ 000_2$$

Hexadecimal Representation

- Compact representation of bit strings
- 4 bits per hex digit

0	0000	4	0100	8	1000	c	1100
1	0001	5	0101	9	1001	d	1101
2	0010	6	0110	a	1010	e	1110
3	0011	7	0111	b	1011	f	1111

CSC258 LEC04 - Numerical Representations

Example

$$eca8\ 6420_{16} = 1110\ 1100\ 1010\ 1000\ 0110\ 0100\ 0010\ 0000_2$$

Practice

- Convert $5ed4$ into a binary number
 - $0101\ 1110\ 1101\ 0100_2$
 - Convert $001\ 111\ 011$ from binary to octal
 - 173_8
 - Convert $1011\ 1110$ from binary to decimal
 - $2^1 + 2^2 + 2^3 + 2^4 + 2^5 + 2^7 = 190_{10}$
 - Convert 35 from octal to hexadecimal
 - Convert 249 from decimal to binary
-

Sign-Magnitude Representation

- Simplest method of representing positive and negative numbers
 - Negative numbers have 1 as a sign bit, where positive numbers have a 0
 - Disadvantages:
 - We can have a positive result for zero, 0000_2 and 1000_2 , which wastes a bit pattern
 - Addition doesn't work the way we want it to ($101_2 + 001_2 = 110_2 \neq 0_2$)
-

2s-Complement Signed Integers

- Range of -2^{n-1} to $2^{n-1} - 1$
- Using 64 bits, the range is $-9,223,272,036,854,775,808$ to $9,223,372,036,854,775,807$
- Top bit is still a sign bit
 - 1 for negative
 - 0 for non-negative
- Non-negative numbers have the same unsigned and signed representations
 - $0 \rightarrow 0000\ 0000 \dots 0000$
 - $-1 \rightarrow 1111\ 1111 \dots 1111$
 - Most positive $\rightarrow 0111\ 1111 \dots 1111$
 - Most negative $\rightarrow 1000\ 0000 \dots 0000$

CSC258 LEC04 - Numerical Representations

- To obtain the magnitude of a negative number, negate each bit and add one to the result
- To convert a positive number into a negative number, negate each bit and add one to the result

Example

$$1111\ 1111\ \dots\ 1111\ 1100_2 = -2^{31} + 2^{30} + \dots + 2^0 = -4_{10}$$

Practice

- Convert 001 111 011 from 2's complement to decimal
 - $2^0 + 2^1 + 2^3 + 2^4 + 2^5 + 2^6 = 123$
 - Convert 1011 1110 from 2's complement to decimal
 - $-1(0100\ 0001_2 + 1_{10}) = -66$
-

Binary Arithmetic and Overflow

- Overflow occurs if the result from an arithmetic operation is too large
 - Data is stored in a fixed and limited number of bits
- Multimedia extensions to standard instruction sets often offer *Saturating Arithmetic* to deal with overflow
 - Cap the results at the max/min predetermined value
- Unsigned integers are commonly used for memory addresses where overflows are ignored
- Compilers can easily check for unsigned overflow using a branch instruction
 - Addition has overflowed if the sum is less than either of the addends

Operation	Operand A	Operand B	Result Indicating Overflow
$A + B$	≥ 0	≥ 0	< 0
$A + B$	< 0	< 0	≥ 0
$A - B$	≥ 0	< 0	< 0
$A - B$	< 0	≥ 0	≥ 0